

Algoritmos y Estructuras de Datos



Integrantes:

- Ignacio Alonso Sclavo
- Marcel
- Jorge Filardi
- Nahuel Vique
- Juan Ignacio Weyrauch

0. Índice

0. Índice.....	1
1. Introducción.....	2
2. Descripción del problema.....	2
3. Modelo de dominio.....	3
- Inventario y terminales.....	3
Responsabilidad del módulo:.....	3
Estructuras utilizadas:.....	3
4. TDAs y estructuras implementadas.....	4
5. Arquitectura y servicios.....	5
6. Algoritmos principales.....	6
7. Pruebas.....	7
8. Complejidad.....	8
9. Conclusiones.....	9

1. Introducción

El proyecto consiste en desarrollar un sistema de gestión para un almacén logístico, aplicando los conceptos vistos en el curso de Algoritmos y Estructuras de Datos.

La aplicación permite registrar productos y stock, gestionar entregas de proveedores, procesar pedidos de reabastecimiento de sucursales y controlar el uso de terminales de carga. Las operaciones principales se apoyan en listas, colas y una cola de prioridad implementadas por el equipo, evitando utilizar las colecciones equivalentes de Java para resolver el núcleo del problema.

2. Descripción del problema

El almacén recibe entregas de productos de distintos proveedores y debe procesarlas respetando el orden de llegada. Las entregas deben esperar y procesarse respetando su orden de llegada; cuando una terminal queda disponible, se asigna la próxima entrega y, al finalizar la descarga, el stock correspondiente se incorpora al inventario.

Por otro lado, las sucursales realizan pedidos de reabastecimiento. Como varias pueden requerir atención al mismo tiempo, el sistema prioriza a la sucursal con mayor cantidad de clientes. Antes de despachar, se verifica que exista stock suficiente para el pedido completo, de forma que una operación rechazada no deje el inventario parcialmente modificado.

Además, las operaciones de carga y descarga dependen de la disponibilidad de las terminales del almacén, que pueden encontrarse libres, ocupadas o deshabilitadas.

De esta manera, el proyecto busca coordinar correctamente el flujo: Proveedor, Almacén, Inventario, Sucursal; el cuál fue el flujo óptimo deducido por el equipo, siempre utilizando las estructuras de datos adecuadas para cada etapa.

3. Modelo de dominio

3.1 Entidades principales

El modelo separa las entidades del negocio de las estructuras utilizadas para almacenarlas. Producto representa cada artículo; ItemInventario relaciona un producto con su stock; Proveedor y Sucursal representan los actores externos; y LineaEntrega y LineaPedido expresan cantidades de productos dentro de una operación.

EntregaProveedor y PedidoReabastecimiento implementan la interfaz OperacionLogistica. Esta decisión permite que TerminalCarga trate ambos casos de manera uniforme: una entrega representa una DESCARGA y un pedido representa una CARGA. La prioridad de un pedido se obtiene de la cantidad de clientes de su sucursal.

3.2. Invariantes relevantes

- Un ItemInventario siempre tiene un producto no nulo.
- El stock de un ítem siempre es mayor o igual a cero.
- En Inventario, no existen dos productos con el mismo código.
- Todo producto registrado tiene código válido.
- Una terminal LIBRE no tiene operación asignada.
- Una terminal DESHABILITADA no tiene operación asignada.
- Una terminal OCUPADA siempre tiene una operación asignada.
- Solo una terminal LIBRE puede pasar a OCUPADA.

4. TDAs y estructuras implementadas

Como primer desafío se implementó una biblioteca propia de estructuras lineales. Los TDAs definen las operaciones esperadas y las clases concretas resuelven su representación.

Estructura	Representación	Uso principal
ListaArray	Arreglo dinámico	Inventario y listado de terminales.
ListaSimple	Nodos enlazados	Líneas de entregas y pedidos y resultados de consultas.
ListaDoble	Nodos con anterior y siguiente	Alternativa de lista con recorrido bidireccional.
ListaCircular / ListaCircularDoble	Enlaces circulares	Implementaciones requeridas en la biblioteca.
Pila	Basada en ListaSimple	Entregas pendientes, preservando FIFO.
Cola	Lista enlazada con referencia al final	Pedidos priorizados por cantidad de clientes y FIFO ante empate.
ColaPrioridad	Lista enlazada ordenada por Comparador	Reutilización de posiciones del arreglo.
ColaCircular	Arreglo circular de capacidad fija	Reutilización de posiciones del arreglo.
Conjunto	Lista sin duplicados	Unión, intersección, diferencia y subconjunto.

En Cola y ColaPrioridad se restringen las operaciones de lista que permitirían insertar o eliminar en posiciones arbitrarias. De esta forma se preserva la semántica FIFO y el orden de prioridad, respectivamente.

5. Arquitectura y servicios

El proyecto se organiza por responsabilidades. El paquete tda contiene los contratos; structures las implementaciones; model las entidades del dominio; service la lógica de negocio; ui la interacción por consola; y Main actúa como punto de entrada de la aplicación.

La clase AlmacenLogistico funciona como coordinadora del sistema. Mantiene el Inventario, las terminales, la Cola de entregas pendientes y la ColaPrioridad de pedidos. De esta forma, la interfaz de usuario no modifica directamente las estructuras internas, sino que solicita operaciones al servicio.

5.1. Consultas definidas para el almacén

Consulta	Justificación
Productos con stock bajo	Detectar posibles faltantes antes de que afecten despachos.
Cantidad total de unidades	Obtener una visión agregada del inventario disponible.
Entrega pendiente por proveedor	Localizar una entrega dentro de la cola de espera.
Pedido pendiente por sucursal	Consultar si una sucursal tiene un pedido aún no despachado.
Terminales por estado	Conocer la disponibilidad operativa de la playa de carga.

Estas consultas requieren recorrer las estructuras del sistema y no se limitan a mostrar un atributo almacenado directamente.

6. Algoritmos principales

6.1. Recepción de proveedores

Las entregas se registran en una Cola. La asignación toma siempre la entrega ubicada al frente y busca la primera terminal LIBRE. Si no hay terminal disponible, la entrega permanece pendiente. Al finalizar la descarga, se recorren sus líneas, se aumenta el stock de cada producto y la terminal vuelve a estado LIBRE.

6.2. Reabastecimiento de sucursales

Los pedidos se insertan en una ColaPrioridad utilizando como criterio la cantidad de clientes de la sucursal. Los pedidos con mayor prioridad quedan antes; cuando dos pedidos tienen la misma prioridad, se conserva su orden de llegada.

Antes de descontar stock se valida el pedido completo. La verificación acumula las cantidades cuando un mismo producto aparece en varias líneas, evitando el caso en que cada línea sea válida por separado pero la suma exceda el stock. Solo después de validar todo el pedido se realizan los descuentos, se retira el pedido de la cola y se asigna a una terminal.

6.3. Gestión de terminales

TerminalCarga encapsula sus cambios de estado. Una operación solo puede asignarse si la terminal está LIBRE; liberar una terminal requiere que esté OCUPADA; y una terminal ocupada no puede deshabilitarse. Esto evita estados inconsistentes y la pérdida de una operación en curso.

7. Pruebas

La solución se validó con JUnit mediante pruebas de las estructuras y pruebas orientadas al problema logístico. La ejecución completa con Maven registra 138 pruebas, sin fallos ni errores.

Grupo	Pruebas
Estructuras lineales	83
Modelo de dominio	10
Inventario	7
TerminalCarga	6
AlmacenLogistico	32
Total	138

Entre los casos borde se probaron estructuras vacías, un único elemento, múltiples elementos, índices inválidos, inserciones y eliminaciones en distintas posiciones, búsquedas, ordenamiento, FIFO, LIFO, prioridad y *wraparound* de la cola circular.

En la solución del almacén se incluyeron escenarios sin terminal libre, stock insuficiente, productos repetidos dentro de un pedido, terminales deshabilitadas y pedidos con igual prioridad.

Un caso especialmente importante verifica que, si el stock total no alcanza, el pedido permanezca pendiente y el inventario no sea modificado parcialmente.

8. Complejidad y optimización

8.1. Desafío 3: oportunidad de optimización

La operación identificada como posible cuello de botella es la búsqueda de productos por código en Inventario. En la versión actual, `buscarItem` recorre linealmente `ListaArray` y tiene costo $O(n)$; este costo se repite también en `obtenerStock`, `aumentarStock`, `disminuirStock` y `hayStock`.

Una alternativa es mantener el inventario ordenado por código y aplicar búsqueda binaria. Esto reduciría las consultas de $O(n)$ a $O(\log n)$, aunque registrar un producto seguiría siendo $O(n)$ porque insertar en una posición intermedia de `ListaArray` requiere desplazar elementos. La mejora resulta especialmente conveniente si el sistema realiza muchas más consultas de stock que altas de productos.

Estado de esta versión: el repositorio revisado todavía utiliza búsqueda lineal. Por ese motivo no se incluyen tiempos experimentales ni se presentan resultados como si la optimización ya hubiera sido implementada. Para completar el Desafío 3 corresponde integrar una alternativa real y medir ambas soluciones sobre los mismos datos.

9. Conclusiones

El proyecto permitió aplicar estructuras lineales a un problema concreto y comprobar cómo la representación elegida condiciona el comportamiento del sistema. La Cola resultó adecuada para preservar el orden de llegada de proveedores, mientras que la ColaPrioridad permitió resolver el criterio de atención de sucursales sin perder FIFO en empates.

La separación entre modelo, estructuras y servicio ayudó a mantener reglas de negocio claras: el inventario protege la consistencia del stock, TerminalCarga controla sus estados y AlmacenLogistico coordina el flujo general. La suite de pruebas permitió detectar y corregir casos borde, especialmente el descuento parcial de pedidos con stock insuficiente.

Como mejora de rendimiento se identificó la búsqueda de productos en inventario. Su optimización constituye el siguiente paso natural para comparar experimentalmente una búsqueda lineal con una alternativa más eficiente.

10. Registro de uso de herramientas de IA

Durante el desarrollo se utilizaron herramientas de IA generativa como apoyo, de acuerdo con lo permitido por la consigna. Su uso estuvo orientado principalmente a revisión, discusión de alternativas y documentación, y no reemplazó la validación del código por parte del equipo.